

**EE272 Homework 2**  
**CNN Accelerator Part 1**  
**Report**

Student Name 1: Berk Gokmen  
Student Name 2: Edison Altamirano

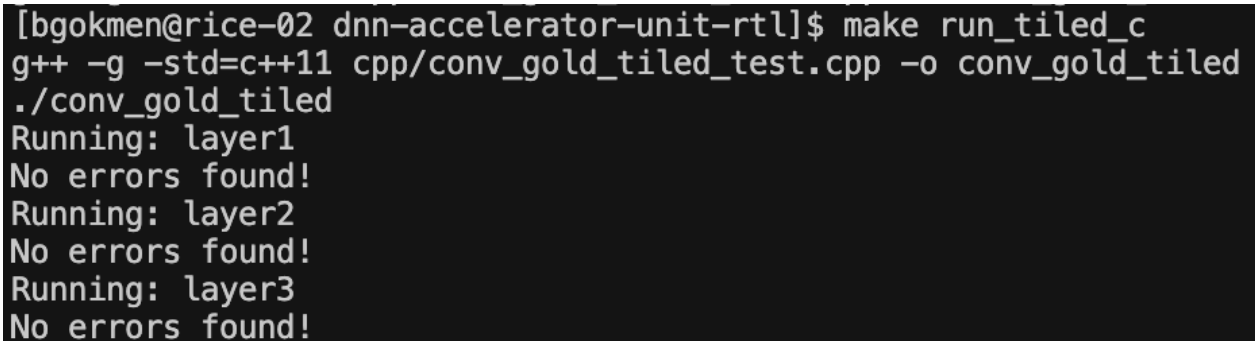
Please answer the questions below in the space provided and include the code/reports indicated in your submission.

1. Please paste a screenshot of passing both *conv\_gold.cpp* and *conv\_gold\_tiled.cpp*. We will run these ourselves as well. (10 points)



```
[bgokmen@rice-02 dnn-accelerator-unit-rtl]$ make run_conv_gold
g++ -g -std=c++11 cpp/conv_gold_test.cpp -o conv_gold
./conv_gold
Running: layer1
No errors found!
Running: layer2
No errors found!
Running: layer3
No errors found!
```

*Figure 1. Screenshot showing conv\_gold.cpp passing tests*



```
[bgokmen@rice-02 dnn-accelerator-unit-rtl]$ make run_conv_gold_tiled
g++ -g -std=c++11 cpp/conv_gold_tiled_test.cpp -o conv_gold_tiled
./conv_gold_tiled
Running: layer1
No errors found!
Running: layer2
No errors found!
Running: layer3
No errors found!
```

*Figure 2. Screenshot showing conv\_gold\_tiled.cpp passing tests*

2. Please paste a screenshot of your design passing all tests. We will run these ourselves as well. (65 points)

```
Running test_accumulation_buffer_tb
Test passed!

Running test_adr_gen_sequential_tb
Test passed!

Running test_adr_gen_sequential_tb_uvm
Test passed!

Running test_aggregator_tb
Test passed!

Running conv_gold_test
Test passed!

Running conv_gold_tiled_test
Test passed!

Running test_deaggregator_tb
Test passed!

Running test_fifo_tb
Test passed!

Running test_fifo_tb_uvm
Test passed!

Running test_ifmap_double_buffer_tb
Test passed!

Running test_ifmap_radr_gen_tb
Test passed!

Running test_mac_tb
Test passed!

Running test_mac_tb_uvm
Test passed!

Running test_ram_sync_1r1w_tb_uvm
Test passed!

Running test_skew_registers_tb
Test passed!

Running test_systolic_array_with_skew_tb
Test passed!

Running test_weight_double_buffer_tb
Test passed!

Tests passed: 17
Tests run: 17
```

Figure 3. Screenshot showing design passing all tests

3. Describe the directed test you wrote in *fifo\_tb.v*. (5 points)

We wrote five directed tests in *fifo\_tb.v*:

- Test 1: Checking reset behavior. This test asserts the *rst\_n* signal and checks the *empty\_n* and *full\_n* flags of the FIFO to ensure that the FIFO is empty.
- Test 2: Checking expected latency. This test is designed to ensure that the enqueue and dequeue operations of the FIFO have the correct latency. To do so, we first enqueue an element to the FIFO. Once the enqueue signal is asserted, in the next clock cycle, we check that the FIFO is no longer empty by checking the *empty\_n* signal. Then, we dequeue the element from the FIFO and check in the next clock cycle that the FIFO is back to its original empty state.
- Test 3: Checking *empty\_n* flag. This test enqueues elements until the FIFO is full, followed by a series of dequeuing elements until the FIFO is empty. Each enqueue and dequeue operation is followed by an assertion on *empty\_n* based on its expected value. Each dequeue operation also checks that the expected data is taken out from the FIFO.
- Test 4: Checking *full\_n* flag. This test enqueues elements until the FIFO is full, followed by a series of dequeuing elements until the FIFO is empty. Each enqueue and dequeue operation is followed by an assertion on *full\_n* based on its expected value. Each dequeue operation also checks that the expected data is taken out from the FIFO.
- Test 5: Checking clear operation. This test enqueues an element to the FIFO and then asserts the *clr* flag to empty the FIFO. In the next cycle, we check the *empty\_n* flag to ensure that the clear operation correctly emptied the FIFO. Each dequeue operation also checks that the expected data is taken out from the FIFO.

4. Describe the directed test you wrote in *input\_double\_buffer\_tb.v* and *weight\_double\_buffer\_tb.v*. (5 points)

The testbenches *input\_double\_buffer\_tb.v* and *weight\_double\_buffer\_tb.v* differ primarily by the parameters used to instantiate the DUT. We set the Weight Double Buffer to a larger depth (*BANK\_DEPTH*=256, *ADDR\_WIDTH*=8) compared to the Input Double Buffer (*BANK\_DEPTH*=128, *ADDR\_WIDTH*=7) to simulate different storage requirements for weight tiles versus input tiles. Besides this, we have one set of regressions for both the input buffer and weight buffer. However, this set of regressions includes access patterns that are unique to the weight and input double buffers respectively. Therefore, we want to clarify that even though we use the same directed tests for both, these directed tests include behaviors that pertain to each as described below:

### Core Functionality Tests:

- Reset test: We verify that asserting the *rst\_n* signal properly initializes the internal state of the double buffer by defaulting to bank 0. This ensures the buffer starts in a known deterministic state before any operations begin.
- Read after write test for bank 0: We perform a directed test where we write specific data patterns to addresses in bank 0 and then read them back. This verifies that when *switch\_banks* is low, the write port correctly targets the first logical bank and the read port retrieves the correct data from it.
- Read after write test for bank 1: We repeat the read-after-write sequence but with *switch\_banks* toggled high. This confirms that the internal address logic correctly flips the MSB to target the second logical bank, verifying that both halves of the underlying SRAM are accessible and isolated.
- Simultaneous read/write test: We assert both *wen* and *ren* simultaneously. This verifies the dual-port nature of the SRAM, ensuring that we can write to one bank while simultaneously reading from the other bank.
- Full memory capacity test: We wrote to the entire memory range for both banks and then read it all back. This confirms that we can successfully use the full capacity of the buffer and that every address is behaving properly up to the maximum depth defined by the parameters.

### Input Double Buffer Tests:

- Instantiated with: *BANK\_DEPTH*=128 and *DATA\_WIDTH*=64.
- Write once, many reads test (overlapping reads): To mimic the data flow of convolution where input pixels are reused across multiple filter applications (sliding window), we wrote a test that writes a data set once and then performs multiple read passes over the same addresses. This verifies that the read operations are not corrupted. We explicitly run this test on both Bank 0 and Bank 1 to ensure that data reuse works correctly regardless of which bank is currently driving the array.

### Weight Double Buffer Tests:

- Instantiated with: *BANK\_DEPTH*=256 and *DATA\_WIDTH*=64.
- Stream of Writes, Stream of Reads Test: We test the ability to burst-write a full tile of weights into one bank while reading a full tile from the other. This mimics how weights are loaded once at the start and then read many times in streams to support reuse. We cycled through the full 256-word depth to confirm that the larger *ADDR\_WIDTH*=8 parameter works correctly and that we can load the next tile simultaneously.

5. Describe the directed test you wrote in *accumulation\_buffer\_tb.v*. (5 points)

We wrote many directed tests in *accumulation\_buffer\_tb.v*:

- Test 1: Checking reset behavior. This test asserts the *rst\_n* signal and checks that the internal bank selection logic defaults to Bank 0. This ensures that the design starts from a known and deterministic state after reset.
- Test 2: Stream test for Bank 0. This test writes a sequence of data to Bank 0 using the primary write port. After the writes complete, we assert the *switch\_banks* signal. We then read the data back through the secondary port (*ren\_wb*) and check that the expected values are returned.
- Test 3: Stream test for Bank 1. This test repeats the stream test for Bank 1. We write a sequence of data to Bank 1, assert *switch\_banks*, and then read the data using the secondary port (*ren\_wb*). Each read checks that the expected data is returned, ensuring that the logic behaves correctly for both banks.
- Test 4: Read-after-write behavior for Bank 0. This test writes known patterns to Bank 0 and immediately reads them back using the primary read port (*ren*). We check that the data appears with the expected 1-cycle latency. After swapping banks, we read the same data from the secondary port (*ren\_wb*) and verify that the values remain correct.
- Test 5: Read-after-write behavior for Bank 1. This test repeats the read-after-write checks for Bank 1. We verify the 1-cycle latency on the primary port (*ren*) and then swap banks and read the data back on the secondary port (*ren\_wb*).
- Test 6: Write once, read many. This test writes a single value to memory and then reads it back ten consecutive times. The test is performed using both the primary read port and the secondary read port. Each read checks that the value remains unchanged, ensuring that reads are not corrupted.
- Test 7: Simultaneous accumulation and read-back. This test fills one bank with data and then swaps banks. While new data is written to the active bank using *wen*, we simultaneously read data from the inactive bank using *ren\_wb*.

6. Describe the UVM test you wrote in *mac\_tb\_uvm.v*. (5 points)

For *mac\_tb\_uvm.v*, we verified the MAC unit using a class-based UVM-style testbench organized around a *mac\_if* interface and the standard components: transaction (*mac\_item*), driver, monitor, scoreboard, environment, and test. The top module instantiates the DUT and connects it through the interface, while the test class launches the environment and generates stimulus for a fixed regression length (*TEST\_LENGTH = 2000*). This structure allows clean separation between stimulus generation, signal driving, observation, and checking.

To stress realistic operating conditions, the test randomizes the main control signals each cycle. In particular, *en* is randomized with a 70/30 distribution, meaning the MAC is frequently enabled but also experiences intermittent disable cycles (random stalls). This directly tests that the MAC only updates outputs when *en=1* and holds its previous outputs when *en=0*. Additionally, *weight\_wen* is randomized with a 10/90 distribution to model occasional weight updates, while *ifmap\_in*, *weight\_in*, and *ofmap\_in* are randomized signed values to exercise a wide range of arithmetic cases.

We implemented a simple golden model inside the scoreboard. The monitor samples the driven inputs at each posedge and sends transactions to the scoreboard, while the driver drives signals on the negedge to ensure setup time before sampling. The scoreboard updates its expected state by latching the weight when *weight\_wen* is asserted and computing  $ofmap\_out = ofmap\_in + ifmap\_in * weight$  when *en=1*. Reset behavior is explicitly checked by forcing the golden outputs to zero when *rst\_n=0* and asserting that the DUT outputs remain zero during reset. Over 2000 randomized cycles, the DUT outputs are compared against the golden model every cycle, providing strong confidence that the MAC handles random stalls correctly and resets properly.

7. Describe the UVM test you wrote in *adr\_gen\_sequential\_tb\_uvm.v*. (5 points)

We began the UVM tests for *adr\_gen\_sequential\_tb\_uvm.v* by implementing the interface, driver, monitor, scoreboard, and environment classes.

The UVM file includes 200 tests in total. In these tests, *adr\_en*, *config\_en*, and *config\_data* are initialized as random bits and fed into the DUT and the golden implementation in the testbench. However, one thing we realized is that randomizing *config\_en* would lead to many resets, since each time *config\_en* is asserted, the output *adr* is set back to 0. To make our UVM test consider a more realistic scenario, we set the distribution of *config\_en* such that it is asserted 5% of the time and de-asserted 95% of the time. Doing so allows the randomized tests to count up to more addresses before being reset by a different configuration. Additionally, to test wrap-around behavior, *config\_data* is restricted to a value between 3 and 4.

Overall, by imposing these restrictions, we ensure that our implementation behaves correctly with random stalls (since *adr\_en* is randomized), has the correct reset behavior (checked with *rst\_n* and *config\_en* signals), correctly counts the addresses (by making sure *config\_en* is de-asserted most of the time), and the edge case of wrap-around behavior is tested (by restricting *config\_data* to a smaller set). This made us confident that our implementation is correct.